

Solo E-Commerce Platform -- Architecture Document

Solo E-Commerce Platform

Document Version 1.0 | March 2026

Document Version: 1.1

Date: 09 May 2026

Author: Solo Engineering Team

Status: Final

1. Introduction

This document is the canonical reference for the architecture of the Solo E-Commerce platform. It describes the macro-level structure (tiers, components, data flow), the cross-cutting concerns (security, observability, deployment), and the conventions that bind them. It complements:

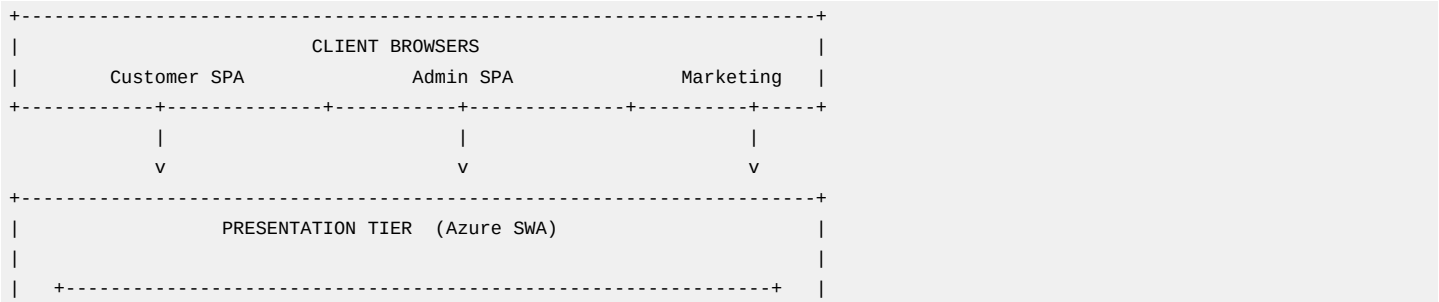
- 01 HLD -- system context & goals
- 02 LLD -- module-level class detail
- 04 Scope & Requirements
- 05 Technical Features

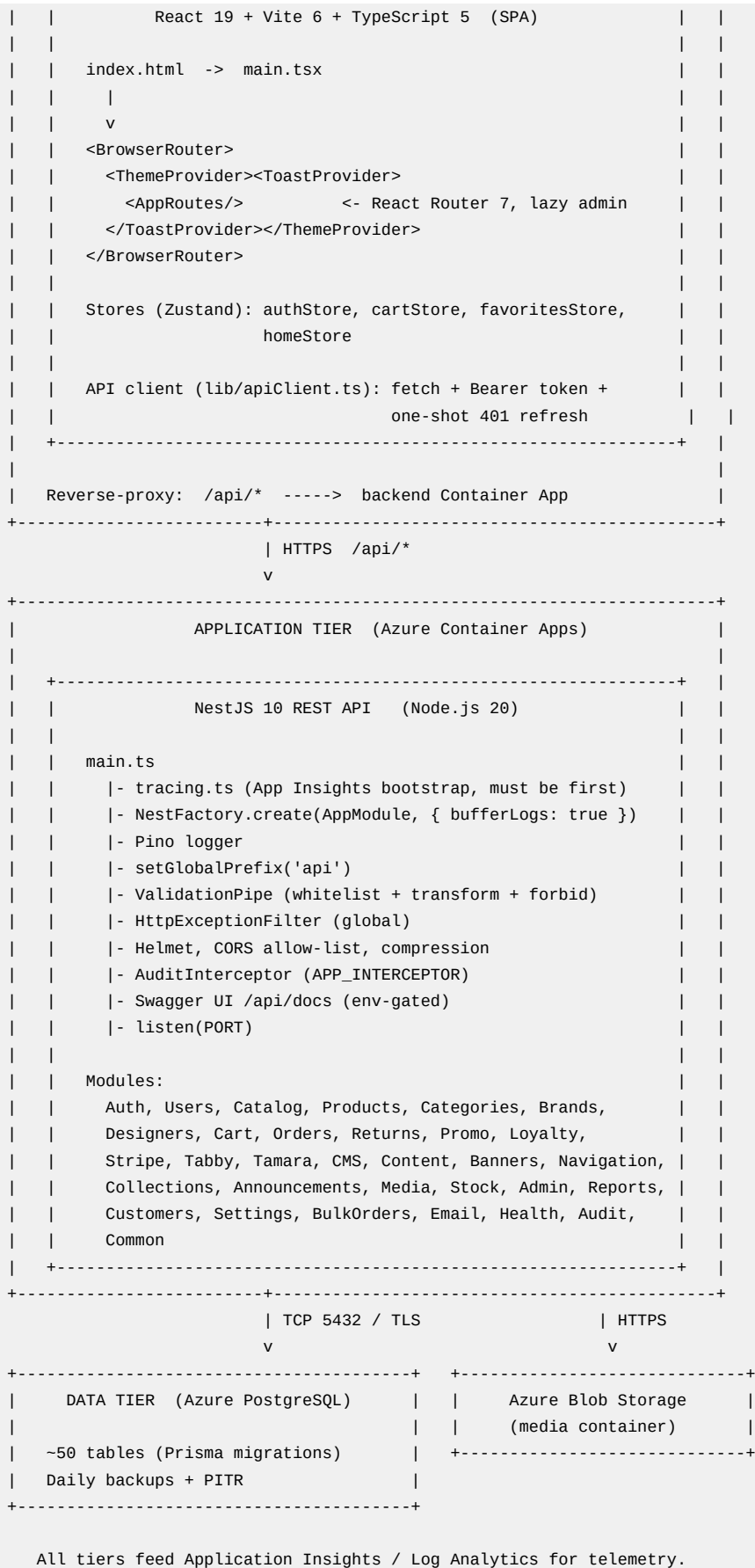
The system is implemented as a headless three-tier web application on Microsoft Azure: a React + Vite SPA frontend, a NestJS REST API backend, and a PostgreSQL database. All inter-tier traffic is HTTPS/TLS, and all business invariants live behind the API.

2. Architectural Principles

#	Principle	Practical implication
P1	Strict tier separation	SPA never speaks to DB; API is the only steward of business state
P2	Type-safety end-to-end	TypeScript on both tiers; Prisma-generated types feed DTOs
P3	Snapshot, don't recompute	VAT, prices, shipping, discount captured on orders at creation
P4	Idempotent integrations	Webhooks dedup via provider event IDs (stripe_events, ...)
P5	Soft-delete by default	deletedAt tombstones on user-visible entities (e.g., products)
P6	RBAC at the controller	RolesGuard + @Roles() on every admin route
P7	Stateless API	All session data in JWT + DB; no in-process user state -- horizontal scale
P8	Append-only audit	audit_logs is write-only via AuditInterceptor
P9	One-command deploy	azd deploy covers backend + frontend
P1	Observability is non-negotiable	Pino structured logs + App Insights traces, correlated by trace ID
P1	Configuration over code	Site settings (VAT rate, shipping tiers) live in site_settings table
P1	Defence in depth	Helmet, CORS allow-list, throttler, validation pipe with whitelist + transform
2		

3. Three-Tier Architecture Overview





Why three tiers? Clear failure boundaries (one tier down does not cascade), independent scaling characteristics (CDN vs CPU vs IOPS), clean security perimeters (the DB is never directly exposed), and

deployment independence (rolling the API does not require redeploying the SPA).

4. Frontend Architecture (React SPA)

4.1 Component Hierarchy

```
App
|- ThemeProvider      (Context -- light/dark)
|- ToastProvider      (Context -- toast notifications)
|- BrowserRouter
  |- AppHeader        (logo, nav, search, cart icon, account menu)
  |- main
    |  |- <Routes>
    |    |- HomePage
    |    |- ProductListPage
    |    |   |- ProductFilters
    |    |   |- ProductGrid
    |    |   |- ProductCard *
    |    |- ProductDetailPage
    |    |   |- ImageGallery
    |    |   |- VariantSelector (color/size/material)
    |    |   |- BuyBox          (price, stock, add-to-cart)
    |    |   |- DescriptionTabs
    |    |- CartPage
    |    |- CheckoutPage      (address, shipping, payment)
    |    |- AccountLayout
    |    |   |- ProfilePage
    |    |   |- AddressesPage
    |    |   |- OrdersPage
    |    |   |- ReturnsPage
    |    |   |- LoyaltyPage
    |    |- AdminLayout        (lazy chunk; RBAC-gated)
    |    |   |- AdminDashboardPage
    |    |   |- AdminProductsPage
    |    |   |- AdminOrdersPage
    |    |   |- AdminCustomersPage
    |    |   |- AdminCmsPage
    |    |   |- AdminAuditLogPage (Security nav group)
    |- AppFooter
```

4.2 Routing (React Router 7)

Path	Component	Auth
/	HomePage	public
/products, /products/:slug	ProductListPage / ProductDetailPage	public
/categories[:slug]	category pages	public
/brands[:slug]	brand pages	public
/landing/:slug	LandingPage	public
/cart, /checkout, /payment-callback	cart & checkout	public/JWT
/favorites	FavoritesPage	JWT
/account/*	account pages	JWT
/auth/*	login, register, reset, verify	public
/admin/*	admin shell (lazy)	JWT + ADMIN
*	NotFoundPage	--

Guarded routes go through `useAuth()` and redirect to

/auth/login?next=... when unauthenticated.

4.3 State Management (Context + Zustand)

- Zustand stores domain state that survives navigation: authStore, cartStore, favoritesStore, homeStore. The authStore persists to localStorage; the others persist only their identity (cart id, favorite ids) and refresh from server on hydrate.
- React Context carries UI-only state: theme (light/dark), toast notifications.
- Stores never call fetch directly -- every network call goes through src/api/<domain>.ts modules that use apiClient.ts.

4.4 API Client Layer (Axios-style fetch wrapper)

Single src/lib/apiClient.ts wraps fetch:

- Base URL from import.meta.env.VITE_API_URL.
- Attaches Authorization: Bearer <accessToken> from authStore.
- On 401 -> calls /auth/refresh once, retries the original request, logs out on failure.
- Decodes server error envelope ({ statusCode, message, error }) into a typed ApiError.

Per-domain modules export typed wrappers (auth.ts, products.ts, cart.ts, orders.ts, admin.ts, ...) so pages call e.g.

adminApi.getAuditLogs(query) rather than touching the raw client.

4.5 Build & Bundle (Vite)

- npm run dev -> Vite dev server at http://localhost:5173 with HMR.
- npm run build -> bundles to dist/ (tree-shaken ESM, code-split per route via React.lazy()) for admin chunk and heavy pages).
- npm run preview -> local prod preview of dist/.
- npm run test -> Vitest (jsdom + RTL).
- ESLint + Prettier on commit.

The admin chunk is lazy-loaded so the customer storefront bundle stays small; first-paint TTI on a 3G profile remains within the NFR budget.

5. Backend Architecture (NestJS)

5.1 Module System

Each business capability is a NestJS module (*.module.ts) that exports a controller + service + DTOs. Modules import their direct collaborators only -- there is no global service registry. The dependency graph is shown in 02 LLD §2.2.

5.2 Cross-Cutting Concerns (Guards, Interceptors, Pipes, Filters)

Concern	Implementation	Scope
Validation	ValidationPipe({ whitelist, transform, forbidNonWhitelisted })	global
Auth	JwtAuthGuard (Passport-JWT)	per-route via @UseGuards()
Optional auth	OptionalJwtAuthGuard	guest carts
RBAC	RolesGuard + @Roles()	admin routes
Throttling	ThrottlerGuard (60/min default; 5/15min on /auth/*)	global
Errors	HttpExceptionHandler (uniform envelope)	global
Logging	nestjs-pino middleware + per-service child loggers	global
Tracing	App Insights SDK (boot-strapped in tracing.ts)	global
Audit	AuditInterceptor (auto-logs admin writes)	global via APP_INTERCEPTOR
Caching	CacheInterceptor on hot reads (catalog landing)	per-controller
Headers	Helmet	global
CORS	Allow-list from CORS_ORIGINS env	global
Compression	compression middleware	global

5.3 Audit Trail System

A NestJS interceptor reads request context (user, IP, UA, route, body) and on success writes a row to `audit_logs`. The action name is derived from the controller method (createProduct -> `PRODUCT_CREATED`) or explicitly set via `@AuditAction('ORDER_REFUNDED')`. Auth events (`AUTH_LOGIN_SUCCESS/FAILED/REGISTERED`) are written explicitly from `AuthService` since they happen outside the standard admin-write flow. The `audit_logs` table is append-only: there is no `UPDATE` or `DELETE` endpoint. Retention is 90 days hot in PostgreSQL with a nightly export to cold storage for the regulatory window.

5.4 Authentication & Authorization Flow

Login	Use API	Refresh	Logout
-----	-----	-----	-----
POST /auth/login	GET /api/...	POST /auth/refresh	POST /auth/logout
verify pwd (argon2)	AuthGuard validates access JWT (HS256)	rotate refresh (issue new pair)	revoke refresh (revokedAt = now)
sign access JWT issue refresh, store hashed	extract role from JWT, RolesGuard checks @Roles()	detect reuse => revoke whole chain	
return tokens	200 / 401 / 403		

Refresh-token reuse detection: every `refresh_tokens` row tracks `replacedByTokenId`; presenting a `revokedAt`-set token causes the chain to be revoked, forcing re-login and surfacing the incident in `audit_logs` as `AUTH_REFRESH_REUSE_DETECTED`.

6. Data Architecture

6.1 PostgreSQL Schema

Approximately 50 tables. Domain groupings:

- Identity: users, refresh_tokens, password_reset_tokens, email_verification_tokens, addresses, saved_payment_methods
- Catalog: products, product_groups, product_images, product_pricing, categories, subcategories, brands, designers, countries
- Cart & Orders: carts, cart_items, orders, order_items, order_status_history, invoices
- Returns: returns, return_items
- Payments: stripe_events, tabby_events, tamara_events
- CMS: home_page_config, home_page_sections, landing_pages, category_landing_pages, banners, navigation_menu_items, product_collections, announcements, media_assets
- Loyalty / Promo: loyalty_wallets, loyalty_transactions, promo_codes, user_promo_usages
- Stock: stock_movements
- Ops: site_settings, bulk_orders, audit_logs, favorites

6.2 Prisma ORM Layer

- Single schema.prisma is the source of truth.
- Prisma-generated types feed DTOs and service signatures.
- Migrations stored under backend/prisma/migrations/ and applied via `npm run prisma migrate deploy` on container start.
- Soft delete handled by adding where: `{ deletedAt: null }` filters in query helpers.
- Critical writes use `prisma.$transaction()` to keep order/stock/loyalty/audit fan-out atomic.

6.3 Migrations

Workflow:

```
# Local dev: edit schema.prisma, then:
npm run prisma migrate dev --name <descriptive_name>

# CI / production:
npm run prisma migrate deploy
```

The Container App start script is

`prisma migrate deploy && node dist/main.js` so a fresh deploy is self-healing.

7. Communication Architecture (REST/JSON over HTTPS)

- Protocol: HTTPS only; SWA + ACA terminate TLS at the edge.
- Format: JSON request/response, UTF-8.
- Versioning: Single major version under /api; breaking changes introduce a new path segment (/api/v2/...) when needed.
- Pagination: Query string page + pageSize -> response shape { items, total, page, pageSize, totalPages }.
- Errors: Uniform envelope { statusCode, error, message, timestamp, path } (see 02 LLD §17).
- Webhooks: Stripe / Tabby / Tamara POST signed payloads to /api/{provider}/webhook; the API verifies HMAC, dedups via provider-event tables, then performs the PAID transition inside a transaction.

8. Security Architecture

Layer	Control
Transport	HTTPS only, HSTS via Helmet
Identity	JWT access (15m) + opaque refresh (7d, hashed at rest)
Passwords	Argon2id (memory-hard, salted)
Input	class-validator on every DTO; global ValidationPipe whitelist
Output	Prisma typed queries (no string concatenation)
Headers	Helmet defaults + CSP recommended on SWA
CORS	Allow-list from CORS_ORIGINS (SWA hostnames only in prod)
Rate limit	Throttler (global 60/min; /auth/* 5/15min)
Webhooks	HMAC signature verification + idempotency tables
RBAC	RolesGuard + @Roles(...) on every admin route
Ownership	Per-resource userId checks before mutation
Secrets	Container App secrets; recommended upgrade to Azure Key Vault
Container	Non-root user, minimal Node 20 alpine base image
DB firewall	ACA subnet + admin IPs only; no 0.0.0.0/0
Image pulls	User-Assigned Managed Identity with AcrPull
Audit	AuditInterceptor writes append-only audit_logs

OWASP Top 10 mapping: see [08 Security & Compliance](./08-security-and-compliance.md#8-owasp-top-10-2021-checklist).

9. Deployment Architecture (Azure)

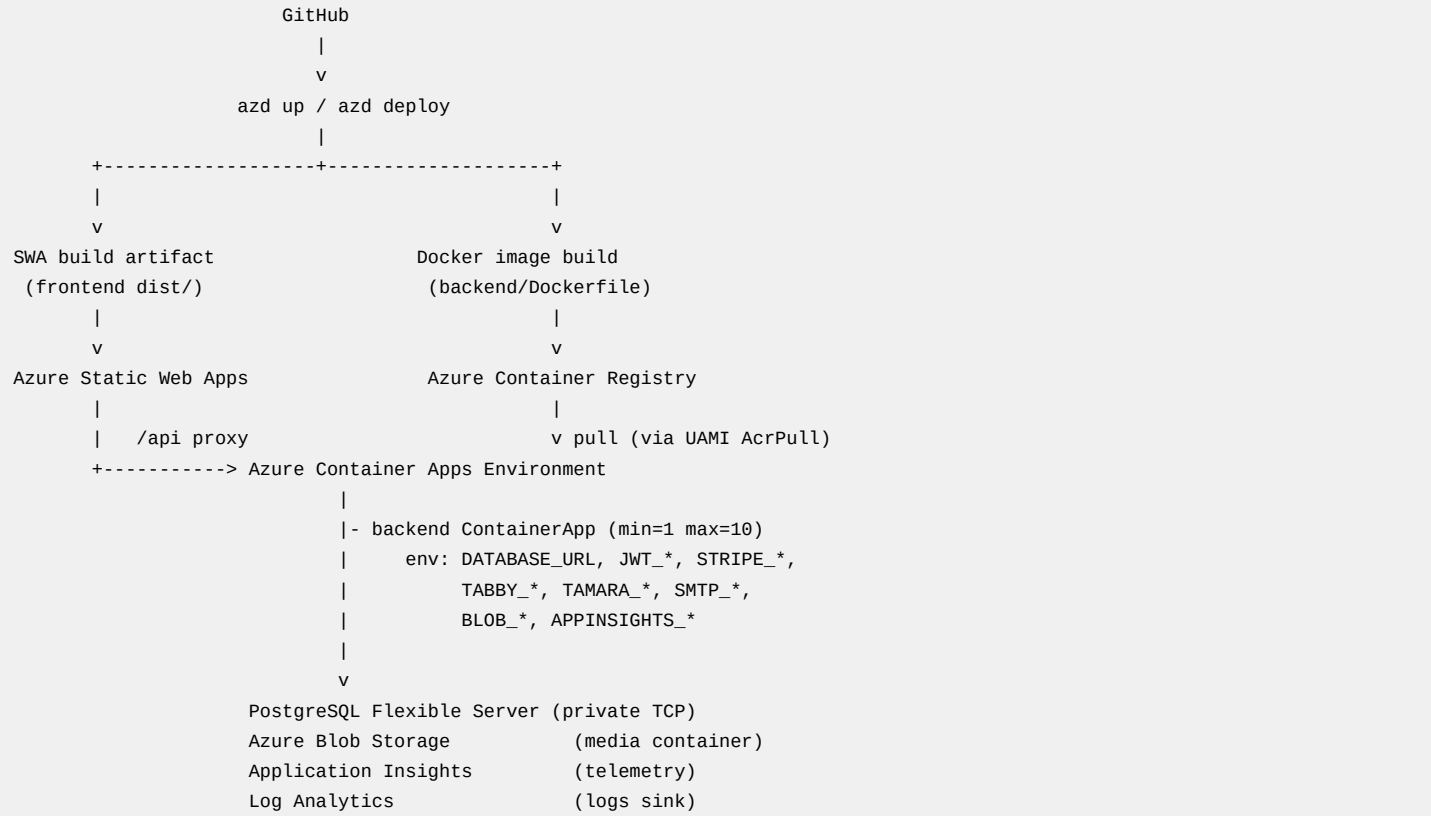
9.1 Development Environment




```
+-----+
| Local PostgreSQL 14      |
| port 5432                |
| (or Docker Compose service)|
+-----+
```

Dev launch: npm run dev in both frontend-react/ and backend/. The SPA's vite.config.ts proxies /api -> localhost:3000 so the production reverse-proxy contract is honoured locally.

9.2 Production Environment



All resources defined in infra/main.bicep with parameters in infra/main.parameters.json.

9.3 CI/CD Pipeline (azd + GitHub Actions)

```
push to main
|
v
GitHub Actions (.github/workflows)
|- npm ci          (frontend + backend)
|- npm run lint
|- npm run build   (frontend -> dist/)
|- npm run build   (backend  -> dist/)
|- npm run test
|- docker build -t backend:<sha> backend/
|- docker push  <acr>/backend:<sha>
|- az containerapp update --image <acr>/backend:<sha>
|- swa deploy     frontend/dist (token from secret)

Routine devops alternatives:
```

```

azd deploy backend      # build, push, update ACA in one step
azd deploy frontend     # build dist, upload to SWA
azd deploy               # both

```

Rolling traffic: ACA shifts 100% to the new revision by default; rollback

= portal or azd containerapp revision activate <prev>.

10. Scalability & Performance

Layer	Strategy
SPA	Static asset edge caching via SWA-managed CDN; lazy admin chunk
API	ACA HTTP-concurrency autoscale (min 1, max 10); stateless containers
DB	Vertical scale on Flexible Server SKU; read-replica plan for catalog reads
Caching	Per-controller CacheInterceptor on hot reads (60s TTL); future Redis for cart sessions and rate-limit counters
Images	sharp-generated thumb/medium/large variants; srcset in markup; Blob -> CDN
Payload	compression middleware; lean DTOs with explicit select
Cold start	min-replicas = 1 plus warm-up endpoint pinged by App Insights availability test

Performance budgets (also in HLD §8):

- API p95 read < 700 ms region-internal
- API p95 write < 1.2 s
- LCP storefront p75 < 2.5 s on 3G
- Cold start container < 5 s

11. Monitoring & Observability

Signal	Source	Sink
HTTP traces	App Insights SDK in NestJS	Application Insights
Structured logs	Pino -> stdout	Container Apps -> Log Analytics
ACA metrics	Built-in	ACA portal + Log Analytics
Postgres metrics	Built-in	Postgres portal
Browser RUM	App Insights JS snippet (optional)	Application Insights
Custom events	AppInsights.trackEvent() (e.g., ORDER_PAID)	Application Insights

Recommended alerts:

- Server errors > 1% of requests for 5 min
- Container restarts > 3 in 10 min
- Postgres CPU > 80% sustained for 10 min
- Stripe webhook failure spike
- App Insights availability test failure

12. Disaster Recovery

Scenario	Recovery Path
DB lost	Restore PIT snapshot to a new Flexible Server, repoint DATABASE_URL, redeploy
Container App lost	azd up reprovisions; ACR holds image digests for rollback
SWA lost	azd deploy frontend redeploys; deployment token stored in env
Region outage	Single-region today; planned: active-passive multi-region (geo-replicate ACR + Postgres + SWA)
Stripe webhook backlog	Stripe auto-retries 3 days with exp backoff; idempotent on our side
Mistaken admin write	Restore from daily backup; cross-check audit_logs to identify scope

Backup posture: Azure-managed daily PG snapshots with PITR; quarterly restore drill recommended (see 09 Deployment & Operations).

End of Architecture Document.